Day 8 - Looking Back, Looking Ahead

Poster work - all model types on a few slides - the history of deep learning in papers - what is new - your posters

Annika Stiehl · Deep Learning · M.Sc. · SRH

Today

Time	
09:00 - 10:00	**Poster work** - last changes, test your laptop on the big screen
10:00 - 11:00	**Recap**: eight days, six model families • which model for which data?
	History in papers, part 1: from the perceptron to AlexNet
11:15 - 12:30	**History, part 2**: ResNet → Transformer → ChatGPT • **new building blocks** • **GNNs**
	New applications • **responsible and explainable AI**
13:30 - 15:30	**Your poster presentations** 🎤
15:30 - 16:00	**Leaderboard** 🏆 • the course on one slide • your feedback

One last Colab example at the end of the recap - otherwise only your poster, your app and your laptop.

09:00-10:00: poster work

🛠️ Last hour for your poster

- finish the poster: **one** message, **one** results table, **one** figure
- numbers on the poster = numbers in your notebook
- practise the talk once - **15 minutes** incl. demo
- decide who says what - everybody speaks

🖥️ Test on the big screen

1. connect your laptop to the projector
2. poster **full-screen** - can you read it from the last row?
3. start your **app** and classify one picture
4. plan B: screenshots of the demo, poster as PDF on a USB stick

I come to every team - show me your poster draft.

Recap: eight days, six model families

10:00-10:40

One recipe for every model

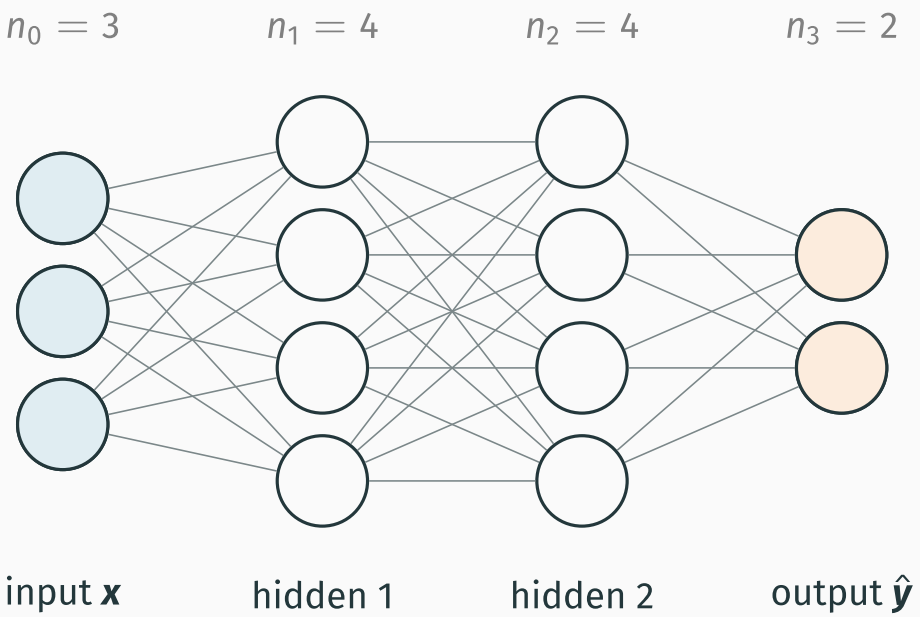
```
for epoch in range(EPOCHS):
    for x, y in train_loader:
        pred = model(x)              # ① the model: changes from day to day
        loss = loss_fn(pred, y)      # ② what is "wrong"?
        optimizer.zero_grad()
        loss.backward()              # ③ backprop: gradients (day 2)
        optimizer.step()             # ④ a small step downhill (day 1)
```

- **days 1-3**: the recipe - loss, gradient descent, backprop, overfitting, metrics
- **days 4-7**: only `model` changes - CNN, RNN, Transformer, autoencoder … · the rest stays: **train / validation / test**, honest metrics

🔑 Key takeaway

A new model is a new `model = ...` line. The loop, the data split and the evaluation stay the same.

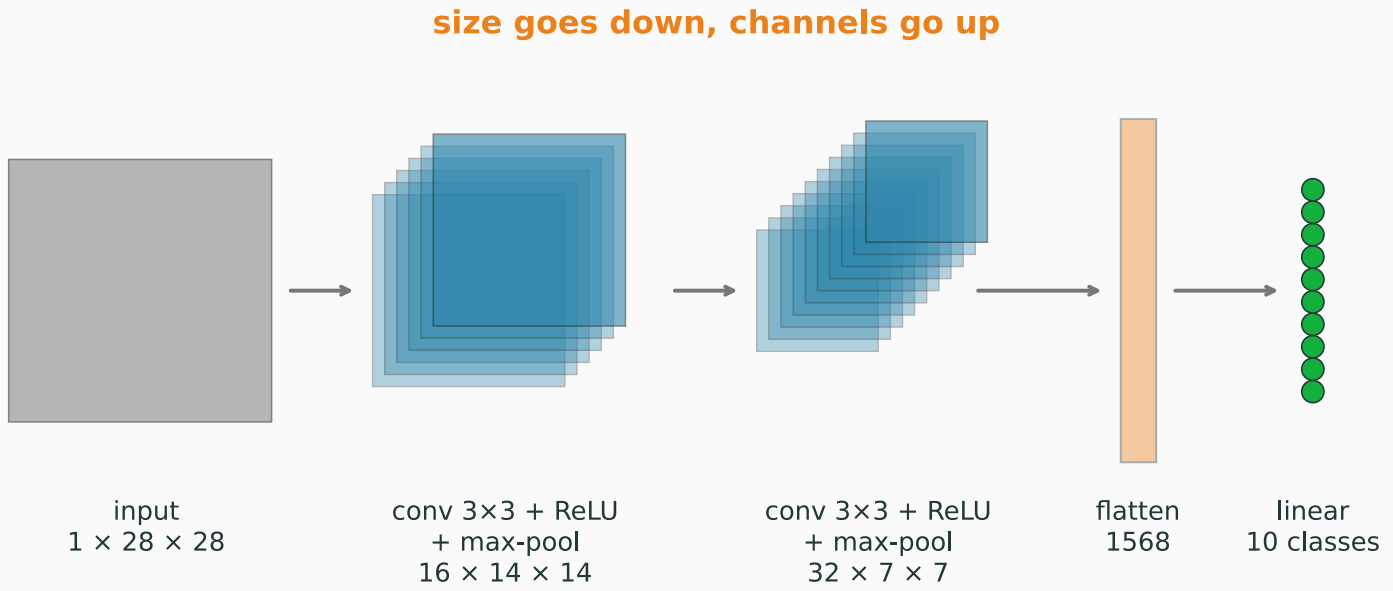
Days 1-2: neuron and MLP - the basic block



- **neuron**: weighted sum + bias → activation (ReLU)
- **MLP**: layers of neurons, every input connected to every neuron
- **input**: a fixed list of numbers (features, a flattened image)
- **idea**: no idea about the data - it must learn everything
- **today**: still inside every big model - the "head" of a CNN, the feed-forward part of a Transformer

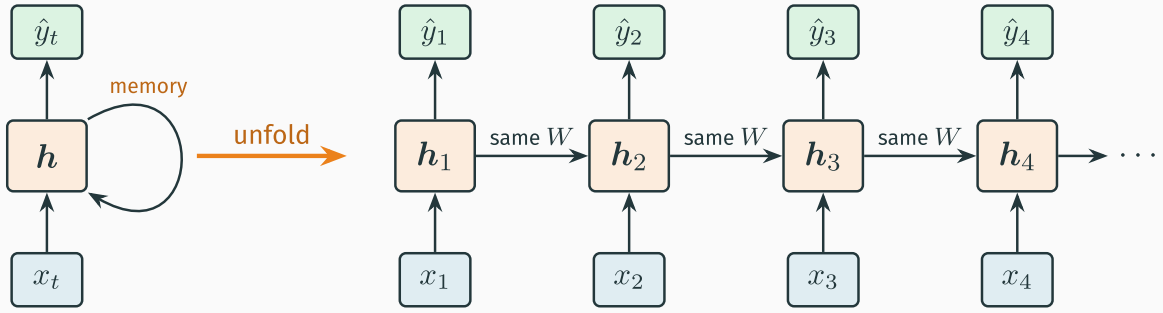
```
nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))
```

Day 4: CNN - for pictures



- a small **filter** slides over the image: the same weights everywhere → finds an edge wherever it is
- **pooling** makes the image smaller; deeper layers see larger parts: edges → shapes → objects
- **input**: images, spectrograms, also 1-D signals (EEG, ECG) · **your challenge**: DermaMNIST

Day 5: RNN, LSTM, GRU - for sequences

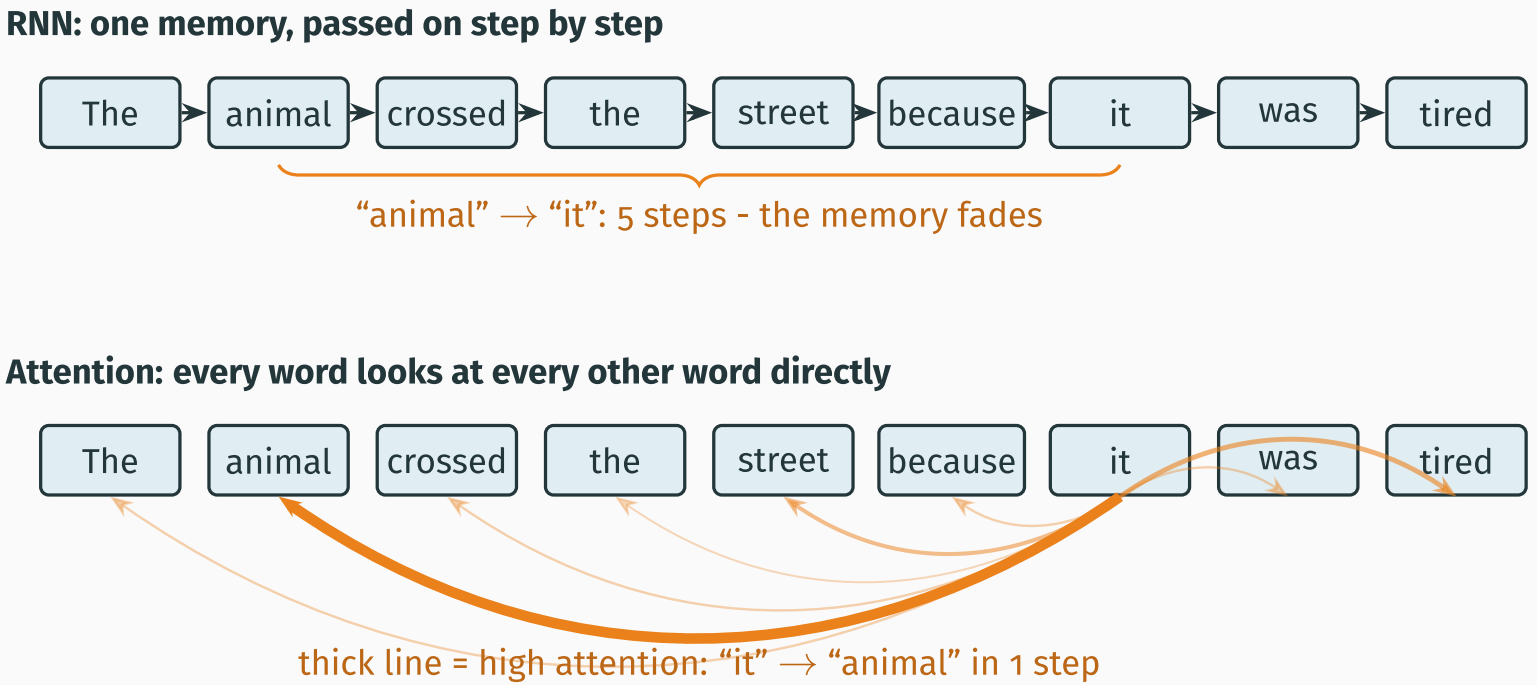


folded

unrolled in time: one copy per time step, all with the same weights

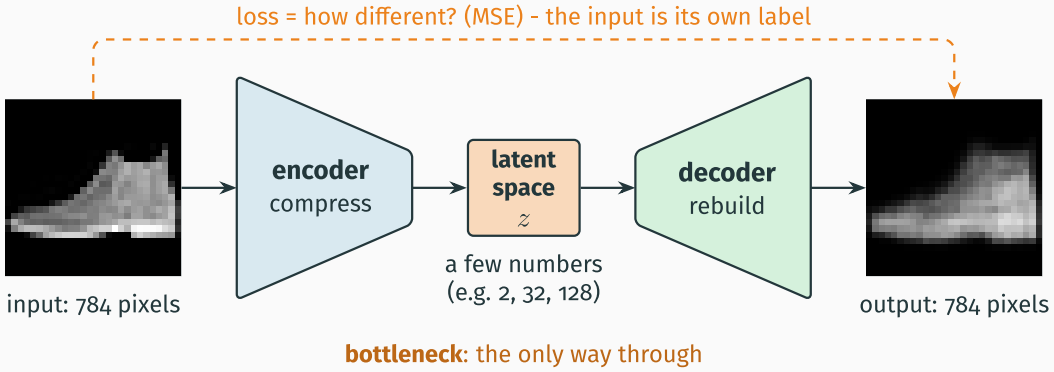
- reads a sequence **step by step**, keeps a **memory** (hidden state)
- the same weights at every time step
- plain RNN forgets quickly (vanishing gradient) → **LSTM / GRU**: gates decide what to keep
- **input**: time series - ECG, EEG, sensor data, text
- slow: step 100 must wait for step 99

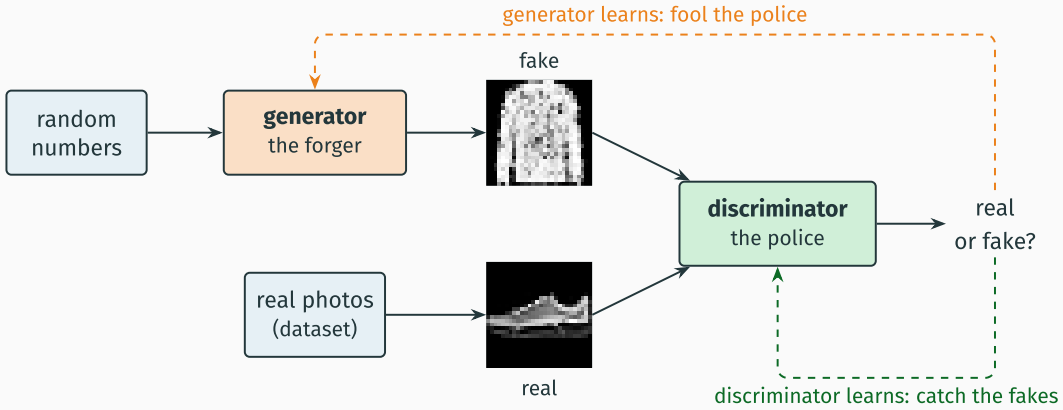
Day 6: Transformer - attention



- every word (token) **looks at every other word** - attention: query, key, value
- all positions at once → fast on GPUs; **positional encoding** tells the order
- **input**: text, image patches (ViT), signals
- **pre-train** on huge data, then **fine-tune**: transfer learning - also what Teachable Machine did (day 1)
- the engine of ChatGPT and all large language models

Day 7: autoencoders and generative models





- **autoencoder**: compress to a small **latent space**, rebuild - no labels needed · denoising, anomaly detection
- **VAE**: a cloud instead of a point → random points give **new** pictures
- **GAN**: generator against discriminator
- **diffusion**: remove noise step by step - Stable Diffusion, DALL·E

Which model for which data?

your data	first try	why
a table of numbers (features)	**MLP** (or a tree model!)	no structure to use
images, spectrograms	**CNN** - pretrained ResNet	neighbouring pixels belong together
time series, short sequences	**LSTM / GRU** or 1-D CNN	order in time
text, long sequences, lots of data	**Transformer** - pretrained	everything looks at everything
no labels, find the unusual	**autoencoder**	large rebuild error = alarm
create new data	**VAE, GAN, diffusion**	learn what the data looks like
a network of things (molecules, electrodes)	**GNN**	the graph is the structure

💻 One last example: CNN + LSTM for heartbeats

```
class CNN_LSTM(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv1d(1, 16, kernel_size=5, padding=2)
        self.conv2 = nn.Conv1d(16, 32, kernel_size=5, padding=2)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool1d(kernel_size=2)
        self.lstm = nn.LSTM(input_size=32, hidden_size=64,
                            batch_first=True)
        self.out = nn.Linear(64, 5)

    def forward(self, x):                          # (batch, 1, 140)
        x = self.pool(self.relu(self.conv1(x)))    # (batch, 16, 70)
        x = self.pool(self.relu(self.conv2(x)))    # (batch, 32, 35)
        x = x.permute(0, 2, 1)                     # (batch, 35, 32)
        outputs, (h, c) = self.lstm(x)             # h: (1, batch, 64)
        return self.out(h[0])                      # (batch, 5)
```

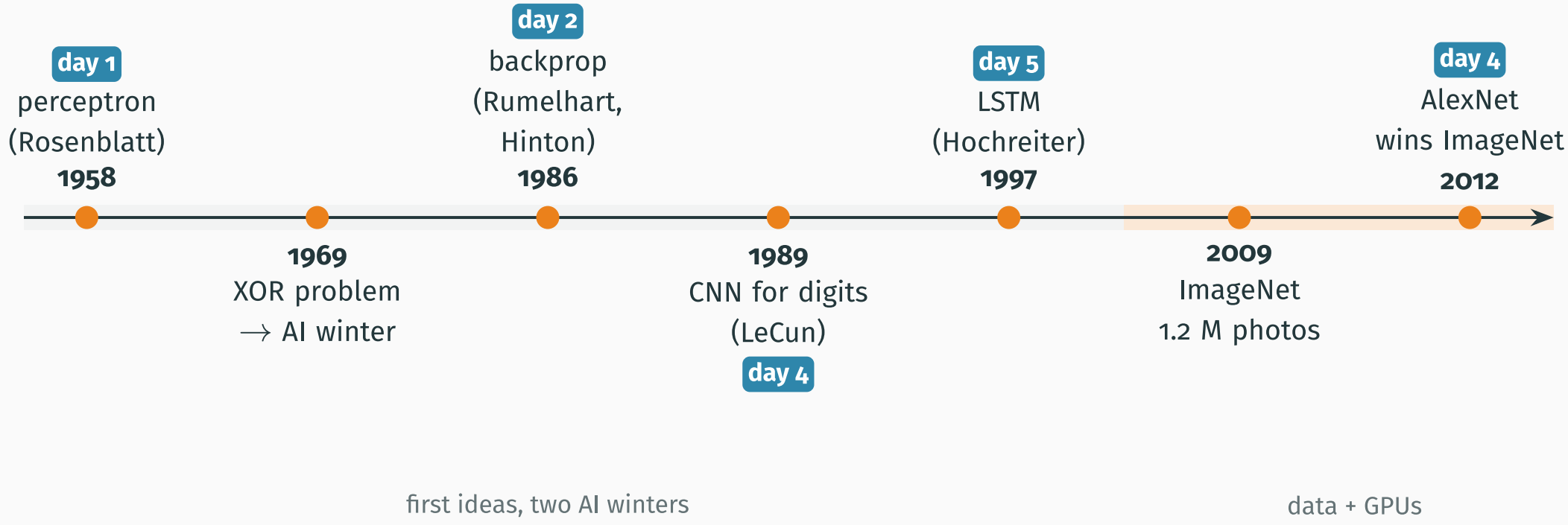
- data: **ECG5000** from Lab 5 - 5 000 heartbeats, 140 time steps, 5 classes
- **CNN** (day 4): finds short patterns, makes the sequence shorter: 140 → 35
- **LSTM** (day 5): reads the 35 steps **in order**
- **Linear**: memory → 5 class scores
- then the **same loop** as always; test set only once

Open in Colab · CPU is enough (~1 min)

A history of deep learning - in papers

10:30-11:00 and after the break

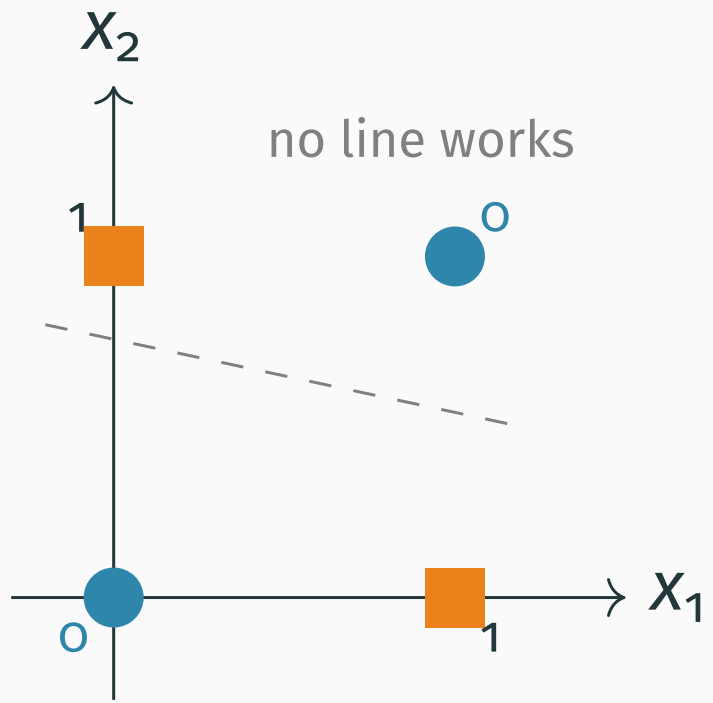
The timeline, part 1: 1958-2012



- blue tags: the day **you** learned it · most ideas are **old** - they needed data and GPUs to work

1958 + 1969: the perceptron - and the first AI winter

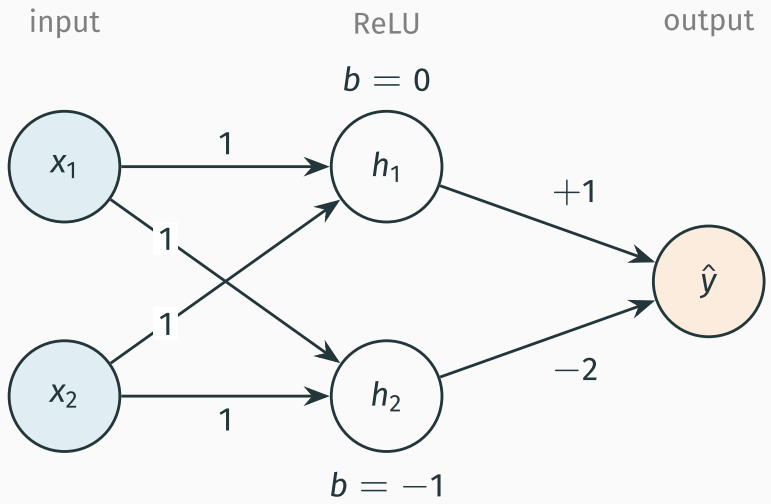
- **Rosenblatt 1958**: the perceptron - one neuron that **learns** its weights from examples
- big promises in the newspapers: machines that walk, talk and see
- **Minsky & Papert 1969**: one neuron cannot solve **XOR** - and multi-layer networks could not be trained yet
- money stops → the first **"AI winter"** (1970s)



Rosenblatt, Psychological Review 1958 · Minsky & Papert, Perceptrons, MIT Press 1969

1986: backpropagation - training many layers

- **Rumelhart, Hinton & Williams 1986** (Nature): the chain rule sends the error **backwards** through all layers
- now hidden layers can learn → XOR solved, MLPs work
- your `loss.backward()` - day 2
- but: computers were slow, data was small → networks stayed shallow; a second winter in the 1990s



Rumelhart, Hinton & Williams, Nature 1986

1989-1997: CNN and LSTM - the building blocks are ready

🖼️ LeCun et al. 1989 / 1998: CNN

- convolution + pooling, trained with backprop
- reads handwritten **zip codes** and **bank cheques** in the US
- LeNet-5 (1998): the blueprint of every CNN

⏳ Hochreiter & Schmidhuber 1997: LSTM

- the memory belt with gates - the RNN can remember for long
- later: speech recognition, Google Translate (2016)

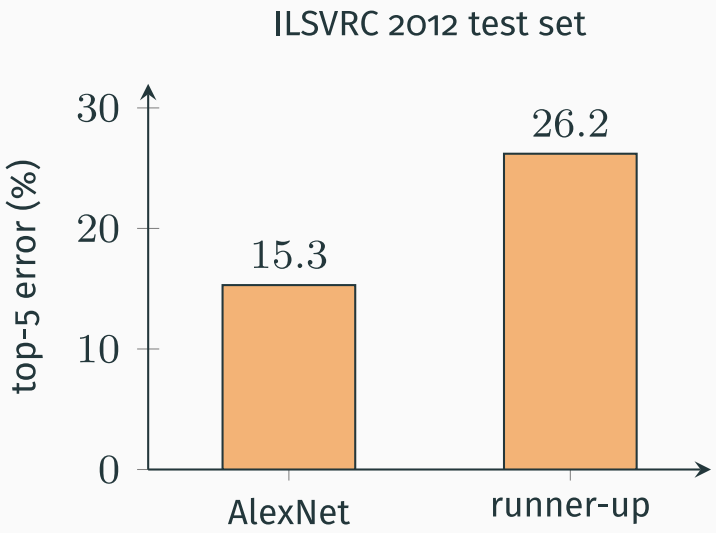
🔑 Key takeaway

By 1997 CNNs and LSTMs existed. What was missing: **big labelled data** and **fast hardware**.

LeCun et al., Neural Computation 1989; Proc. IEEE 1998 · Hochreiter & Schmidhuber, Neural Computation 1997

2009 + 2012: ImageNet and AlexNet - the big bang

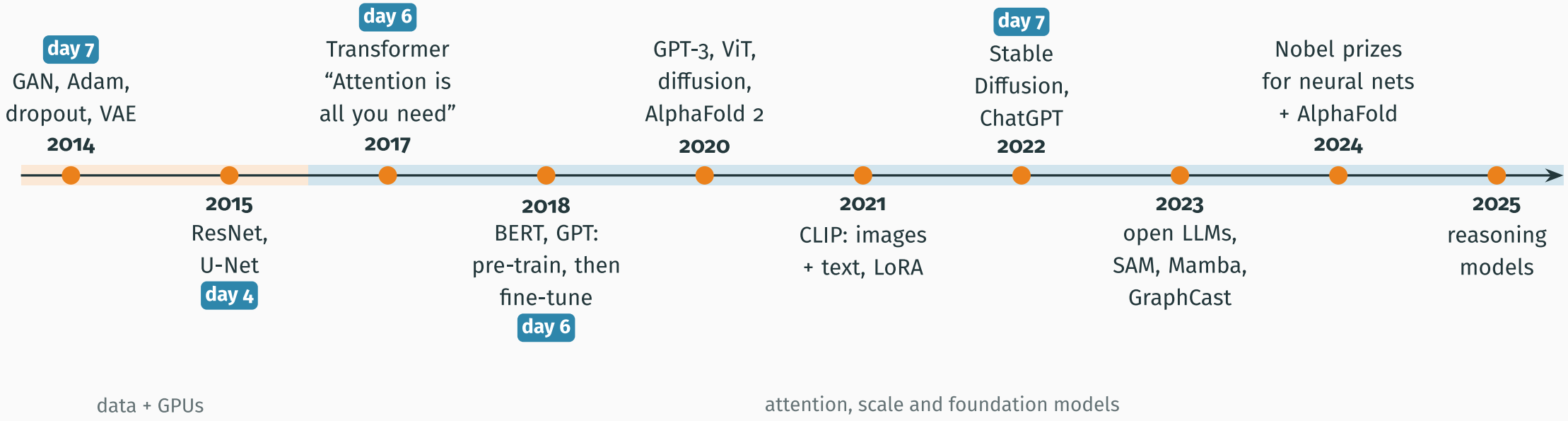
- **ImageNet** (Fei-Fei Li et al., 2009): 1.2 million labelled photos, 1000 classes - a yearly competition
- **AlexNet** (Krizhevsky, Sutskever & Hinton, 2012): a CNN with 8 layers, trained on **two gaming GPUs**
- error 15 % instead of 26 % - nobody had expected such a jump
- the ingredients you know: **ReLU** (day 2), **dropout** + **augmentation** (day 3), **GPU**
- after 2012: almost the whole field switches to deep learning



Deng et al., CVPR 2009 · Krizhevsky, Sutskever & Hinton, NeurIPS 2012

History, part 2: from ResNet to ChatGPT

The timeline, part 2: 2014-2025



- the pace has changed: from one big idea per decade (part 1) to **several per year**
- all five blue tags are from days 4-7 of this course

2013-2014: a wave of ideas

🧮 Training

- **Adam** optimizer (Kingma & Ba, 2014)
- **dropout** (Hinton et al. 2012, Srivastava et al. 2014)
- `optim.Adam(...)`, `nn.Dropout(...)`

🎨 Generating

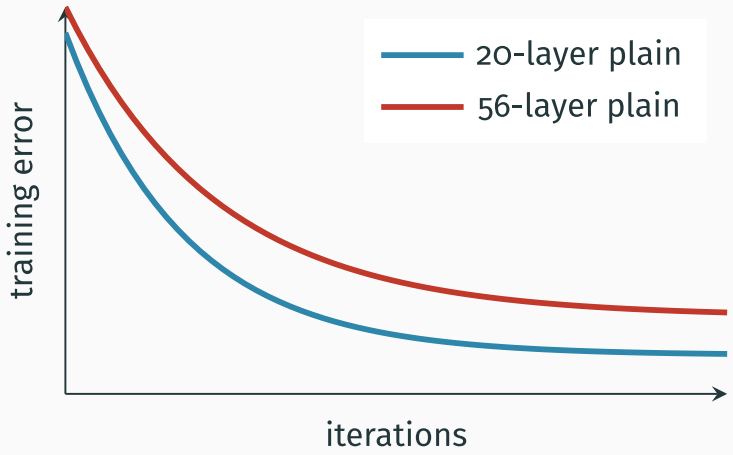
- **VAE** (Kingma & Welling, 2013)
- **GAN** (Goodfellow et al., 2014)

🗣️ Language

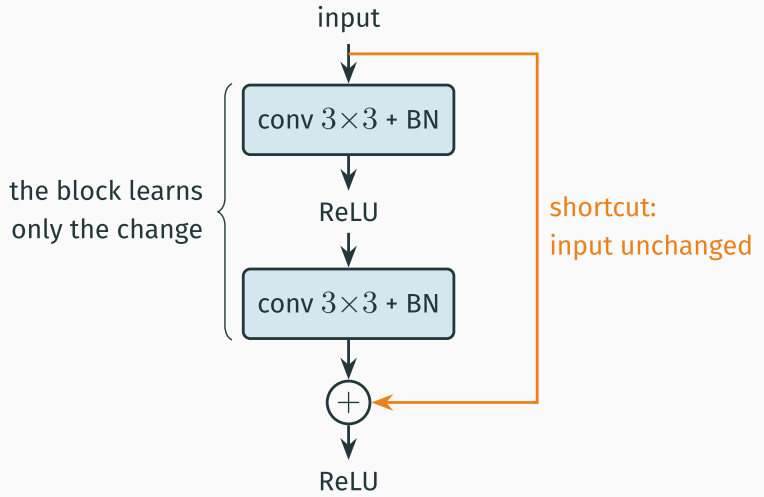
- **word2vec** (Mikolov et al., 2013): words as vectors
- **seq2seq** + first **attention** (Bahdanau et al., 2014): translate with an RNN
- → the way to the Transformer

Kingma & Ba, arXiv 2014 / ICLR 2015 · Hinton et al., arXiv 2012 · Srivastava et al., JMLR 2014 · Kingma & Welling, ICLR 2014 · Goodfellow et al., NeurIPS 2014 · Mikolov et al., 2013 · Bahdanau et al., ICLR 2015

2015: ResNet - why deeper was worse, and the shortcut



schematic, after He et al. (2016)

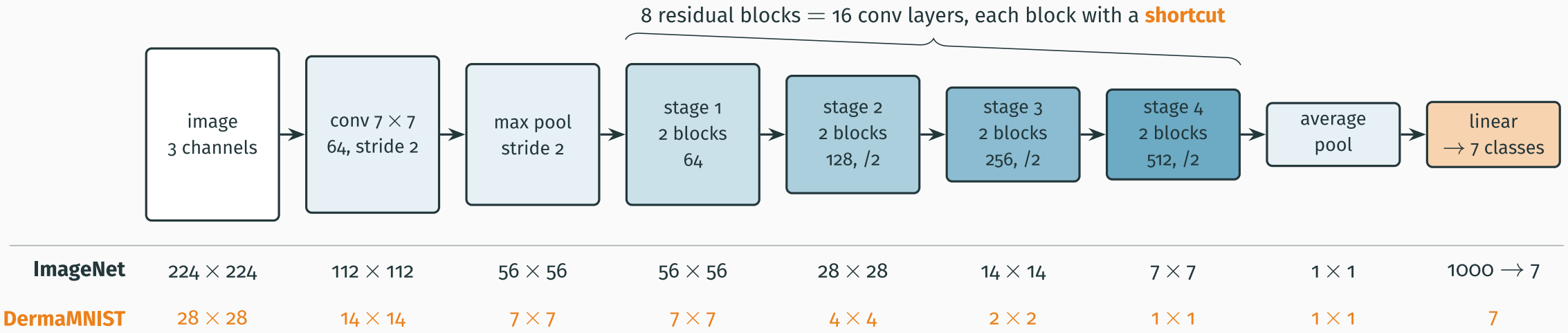


output = ReLU(change + input)

- the problem (day 4): a 56-layer CNN was **worse** than a 20-layer one - even on the **training** data
- the fix: a **shortcut** adds the input to the output: output = input + change
- a block that is not needed learns change ≈ 0 → an extra block cannot make things worse
- the gradient flows back through the shortcut → **152 layers**, 3.6 % error on ImageNet (humans: ~5 %)

He, Zhang, Ren & Sun (Microsoft), "Deep residual learning for image recognition", CVPR 2016 (arXiv Dec 2015)

Inside a ResNet-18 - and in your challenge

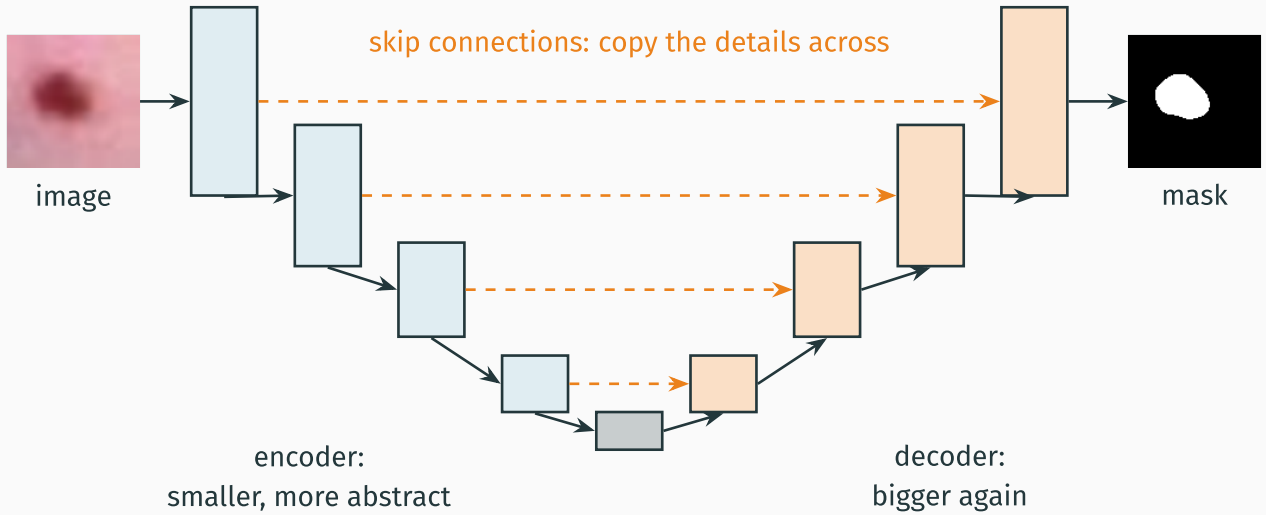


model	layers	parameters	block
ResNet-18	18	11.7 M	2 × conv 3×3
ResNet-34	34	21.8 M	2 × conv 3×3
ResNet-50	50	25.6 M	1×1 → 3×3 → 1×1 ("bottleneck")
ResNet-152	152	60.2 M	bottleneck

For 28 × 28 pictures

- the map shrinks to **1 × 1** after stage 4 → **upsample** the picture (e.g. to 224) or a smaller first layer (3 × 3, stride 1, no max pool)
- replace the last layer: 1000 → **7** classes
- three teams used ResNet blocks

2015: U-Net - a label for every pixel



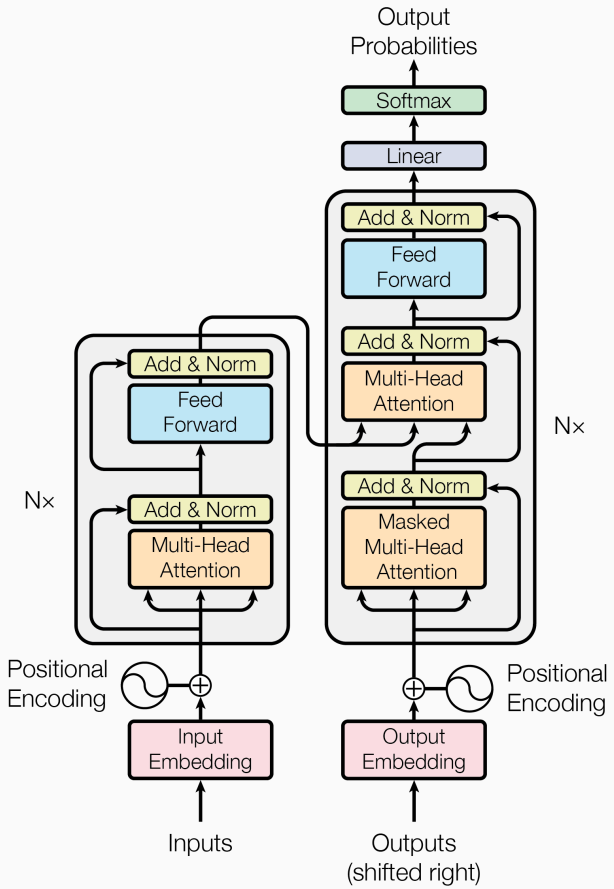
- **segmentation**: not "melanoma yes / no" but **where** is the lesion? → a mask
- **encoder**: a normal CNN - "what is it?"
- **decoder**: back to full size - "where is it?"
- **skip connections** copy the details across

- trained on only **30** images (+ heavy augmentation)
- tumours, organs, cells - **nnU-Net** (2021) still wins many challenges
- inside **Stable Diffusion**: the U-Net removes the noise

Ronneberger, Fischer & Brox (Freiburg), MICCAI 2015 · Isensee et al., "nnU-Net", Nature Methods 2021 · images: DermaMNIST (Yang et al., Sci. Data 2023), CC BY-NC 4.0

2017-2018: the Transformer - and pre-training

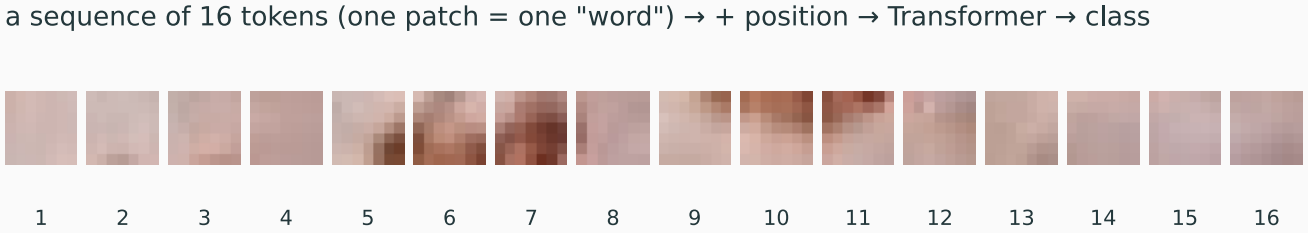
- **Vaswani et al. 2017**: "Attention is all you need" - no RNN, only attention → fast on GPUs - day 6
- **BERT** (Google, 2018): read huge amounts of text, fill in hidden words → fine-tune for any task
- **GPT** (OpenAI, 2018): predict the **next word** - again and again
- the new recipe: **pre-train once** on huge unlabelled data, **adapt** to many tasks



Vaswani et al., NeurIPS 2017 (figure: Google permission for scholarly use) · Devlin et al., NAACL 2019 · Radford et al., OpenAI 2018

2020: scale - GPT-3 and the Vision Transformer

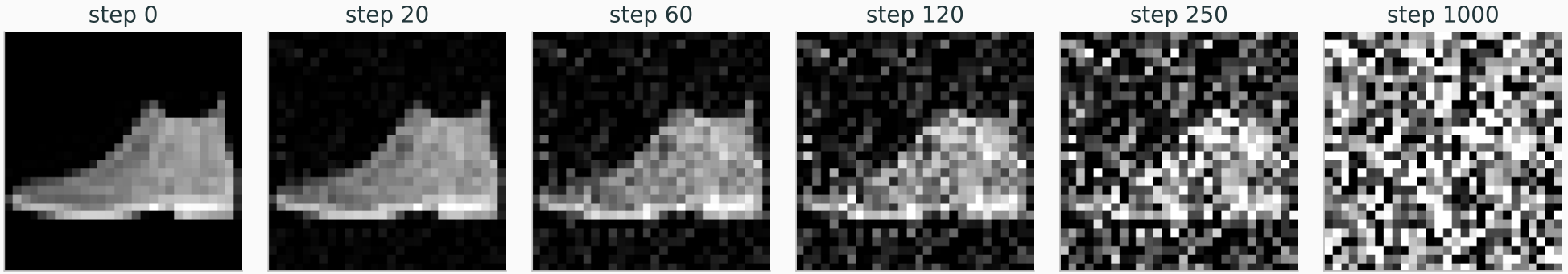




- **GPT-3** (2020): 175 **billion** weights - can solve tasks it was never trained for, from a few examples in the prompt
- **scaling laws**: more data + more weights + more compute → the loss goes down **predictably**
- **ViT** (2020): cut an image into 16 × 16 patches = "words" → a Transformer for images
- with enough data, ViT beats CNNs; with little data, CNNs are often still better

Brown et al., NeurIPS 2020 · Kaplan et al., arXiv 2020 · Dosovitskiy et al., ICLR 2021

2020-2022: diffusion and CLIP - pictures from text



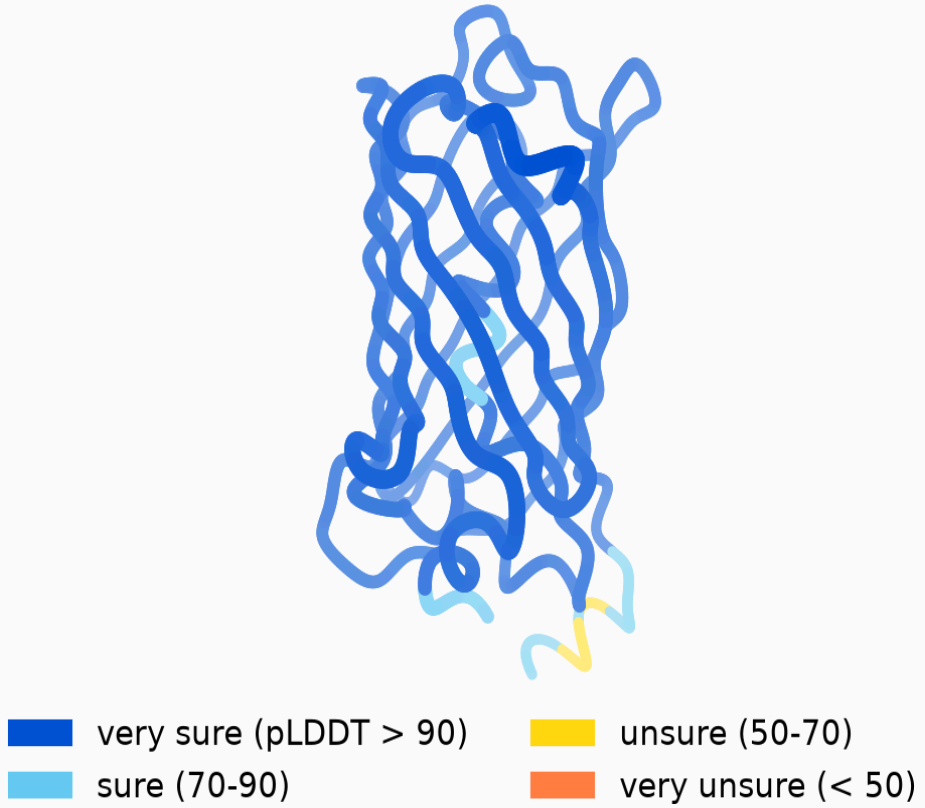
→ forward: add a little noise, 1000 times (no learning) ← backward: a network removes the noise, step by step

- **DDPM** (Ho et al., 2020): diffusion - learn to remove noise step by step (day 7)
- **CLIP** (OpenAI, 2021): images and texts in **one latent space**
- **Stable Diffusion** (Rombach et al., 2022): diffusion in a small latent space + text from CLIP → "a photo of an astronaut riding a horse"
- open weights → anyone can run it on a gaming GPU

Ho, Jain & Abbeel, NeurIPS 2020 · Radford et al., ICML 2021 · Rombach et al., CVPR 2022

2020-2024: AlphaFold - deep learning wins a Nobel prize

- proteins fold into a 3-D shape - the shape decides what they do
- measuring one shape in the lab: months to years
- **AlphaFold 2** (DeepMind, 2020): predicts the shape from the sequence of amino acids - almost as precise as the lab (CASP14)
- a database with **200 million** predicted protein shapes - free for everyone
- **Nobel prize 2024**: Chemistry → Hassabis, Jumper (AlphaFold) + Baker · Physics → Hopfield + **Hinton**



GFP (makes jellyfish glow), predicted by AlphaFold

Jumper et al., Nature 2021 · Varadi et al., Nucleic Acids Research 2024 (AlphaFold DB) · nobelprize.org 2024 · structure: AlphaFold DB AF-P42212-F1 (CC BY 4.0), drawn by us

2022-2025: ChatGPT and the large language models

- **InstructGPT / ChatGPT** (2022): GPT + training with **human feedback** (RLHF) → follows instructions, chats
- 100 million users in two months
- **2023**: GPT-4 also reads images; **open-weight models** (LLaMA, Mistral, ...) - run them yourself
- **2024-2025**: **reasoning models** (OpenAI o1, DeepSeek-R1): "think" step by step before they answer → much better at maths and code
- the core is still the Transformer from day 6, predicting the next token

⚠️ Still true

LLMs can **make things up** (hallucinate) - with full confidence. Check facts, sources and code. For your report: say how you used AI.

Ouyang et al., NeurIPS 2022 · OpenAI 2023 (GPT-4) · Touvron et al., 2023 (LLaMA) · Guo et al. (DeepSeek-AI), Nature 2025 (R1)

Deep dive: what changed - and what did not

📖 Deep dive · self-study

	1998 (LeNet)	2012 (AlexNet)	2020 (GPT-3)	2024-25 (frontier LLMs)
weights	60 thousand	60 million	175 billion	hundreds of billions to trillions (often not published)
training data	60 000 digits	1.2 million photos	~300 billion tokens	trillions of tokens
hardware	a workstation CPU	2 gaming GPUs, ~1 week	thousands of GPUs	tens of thousands of GPUs

Scaling laws (Kaplan et al. 2020; Hoffmann et al. 2022, "Chinchilla"): the test loss falls like a power law in model size N, data D and compute C, e.g. $L(N) \approx (N_c/N)^{\alpha_N}$ with $\alpha_N \approx 0.076$. Chinchilla: for a fixed compute budget, grow data and model **together** - about 20 tokens per weight.

What did not change: gradient descent, backprop, a loss, train / validation / test. A 2025 LLM is trained with the same loop as your challenge model - with the loss "cross-entropy of the next token".

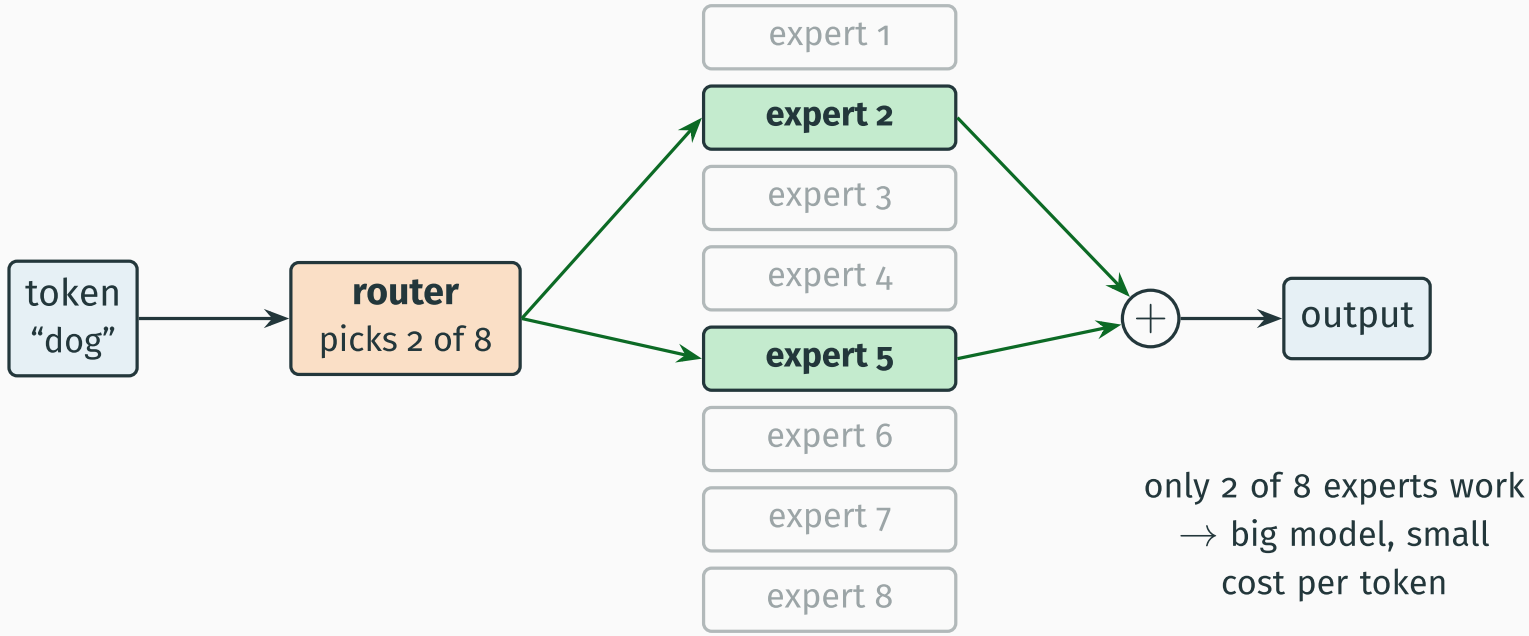
Kaplan et al., arXiv:2001.08361 · Hoffmann et al., NeurIPS 2022 · Brown et al., NeurIPS 2020

New building blocks

The building blocks - old and new

building block	idea	since	where
fully connected layer, ReLU	weighted sum + bend	1958 / 2010	everywhere
convolution	the same small filter everywhere	1989	CNN, U-Net
recurrence + gates	a memory along the time steps	1997	LSTM, GRU
shortcut (residual)	add the input to the output	2015	ResNet, every Transformer
attention	every token looks at every token	2017	Transformer
mixture of experts	only a few sub-networks work per token	2017 / 2021	large LLMs
state space model	an RNN that trains in parallel	2023	Mamba, long sequences
LoRA adapter	fine-tune with a tiny add-on	2021	adapting LLMs and image models
contrastive encoders	two encoders, one shared latent space	2021	CLIP, search, medical image + report
message passing	nodes talk to their neighbours	2009 / 2017	GNNs

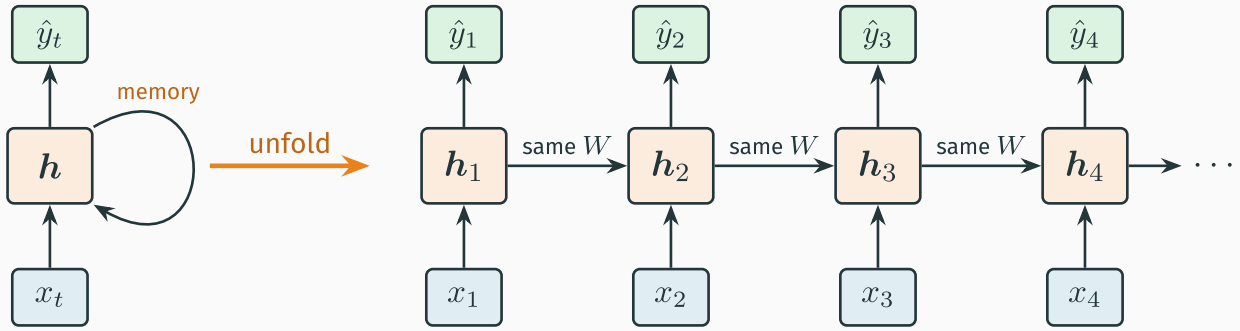
Mixture of experts: a big model that works like a small one



- the feed-forward part of a Transformer block is copied into many **experts**
- a small **router** sends each token to only 1-2 experts
- many weights in total (knowledge) - but only a few are used per token (speed)
- used in many large LLMs, e.g. Mixtral 8 × 7B, DeepSeek-V3

Shazeer et al., ICLR 2017 · Fedus, Zoph & Shazeer, JMLR 2022 (Switch Transformer) · Jiang et al., arXiv 2024 (Mixtral)

State space models (Mamba): the RNN is back



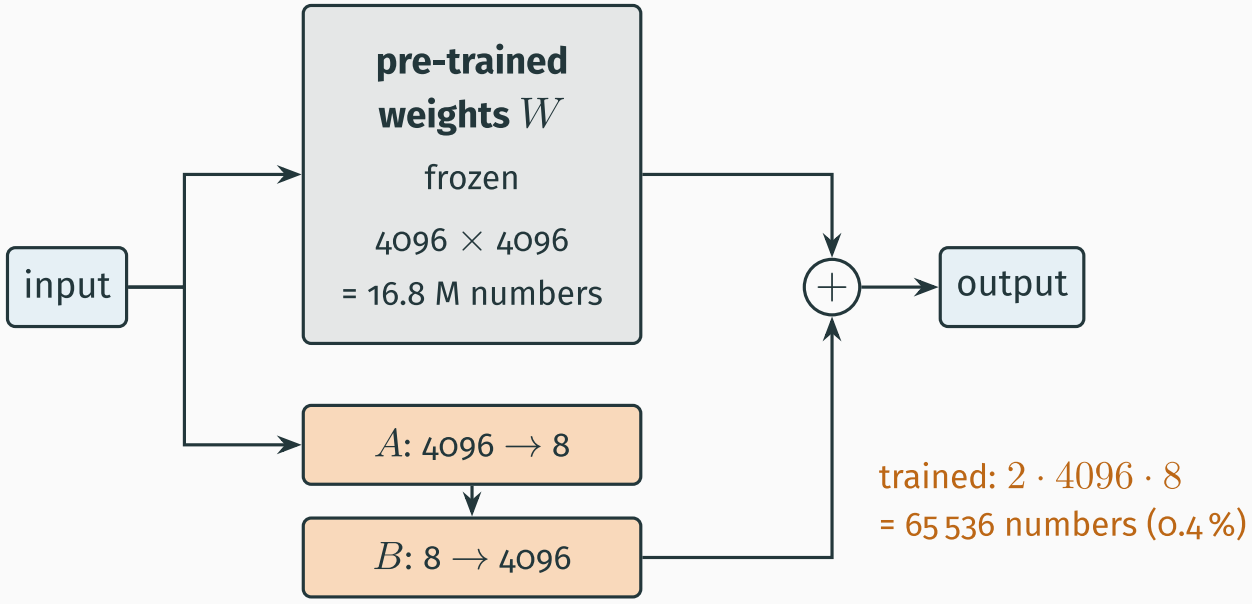
folded

unrolled in time: one copy per time step, all with the same weights

- attention: every token looks at every token → 10× longer text = **100× more work**
- an RNN only carries a memory: 10× longer = 10× more work - but slow to train
- **Mamba** (Gu & Dao, 2023): a memory like an RNN, but trainable **in parallel** like a Transformer; the model chooses what to keep
- good for very long sequences: DNA, audio, hours of EEG
- often **mixed** with attention layers in one network

Gu & Dao, "Mamba", COLM 2024 (arXiv:2312.00752)

LoRA: fine-tune a giant with a tiny add-on



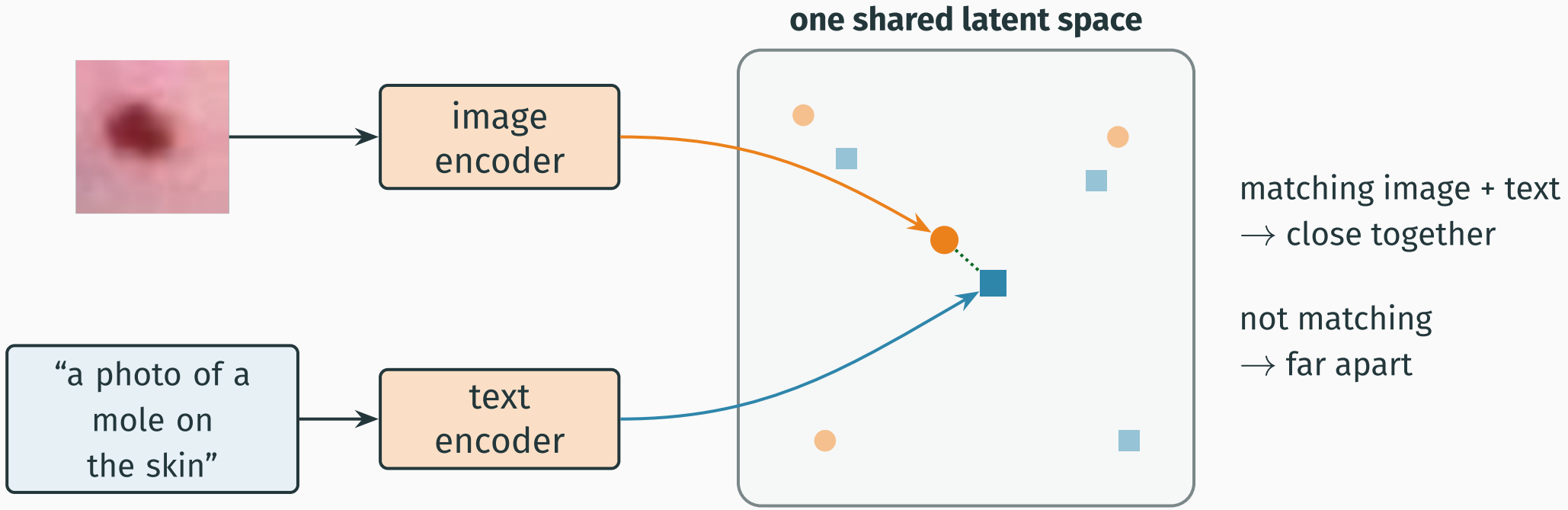
- the big weight matrix stays **frozen**
- next to it: two thin matrices A and B - only these are trained
- here: **0.4 %** of the numbers per layer
- one big model + many small LoRA files: one per hospital, per language, per style

🔑 Key takeaway

Transfer learning (day 6) for models that are too big to fine-tune: freeze almost everything, train a small add-on.

Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022

CLIP: pictures and words in one latent space



- trained on 400 million image-text pairs from the internet: matching pairs **close**, others **far**
- **zero-shot**: classify without training - which text is closest to the picture?
- the text part of Stable Diffusion; search engines for images; in medicine: X-ray + report

Radford et al., ICML 2021 · thumbnail: DermaMNIST (Yang et al., Sci. Data 2023), CC BY-NC 4.0

Graph neural networks: when the data is a network

🧪 Molecules

atoms = **nodes**, bonds = **edges**
→ is it toxic? a new drug?

🧠 EEG

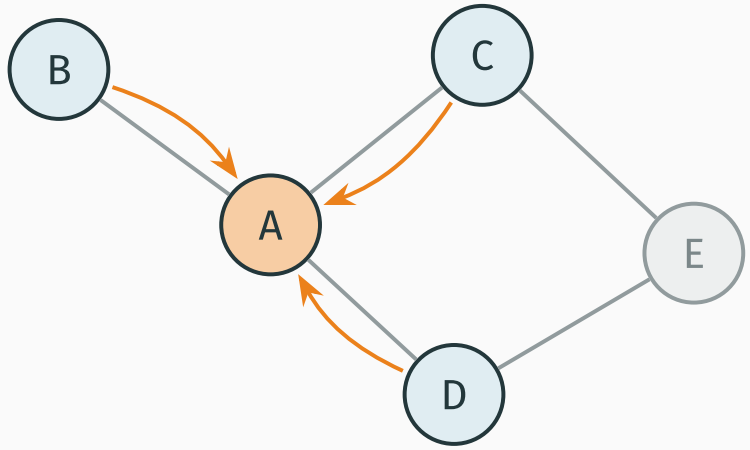
electrodes = nodes, neighbours or correlation = edges → seizure?

🌍 Weather, maps, people

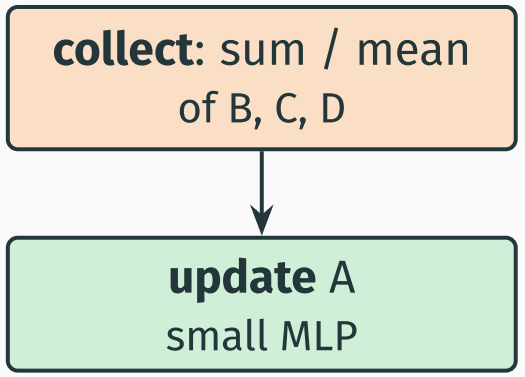
points on the globe, roads, friends in a social network

- a graph has **no grid** (CNN) and **no order** (RNN): every node can have a different number of neighbours
- the GNN works on the graph as it is - like a CNN works on the pixel grid

The GNN idea: message passing



1) every neighbour sends a message to A



2) A gets a new vector

3) all nodes do this at the same time = **one GNN layer**

2 layers: A also hears from E (2 steps away)

same weights for every node → any graph size

- like a CNN: the same small network for every node → works for 10 or 10 000 nodes
- a classic: **GCN** (Kipf & Welling, 2017) · **GAT** (2018) = attention over the neighbours
- **GraphCast** (Google DeepMind, 2023): a GNN predicts the weather 10 days ahead - better than the classic model on most measures

Kipf & Welling, ICLR 2017 · Veličković et al., ICLR 2018 · Lam et al., Science 2023 (GraphCast)

Deep dive: GCN in one formula

📖 Deep dive · self-study

One GCN layer turns node vectors $H^{(l)}$ (one row per node) into $H^{(l+1)}$:

$$H^{(l+1)} = \mathrm{ReLU}\left(\hat{A}\, H^{(l)}\, W^{(l)}\right), \qquad \hat{A} = \tilde{D}^{-1/2}(A + I)\,\tilde{D}^{-1/2}$$

- A: adjacency matrix ($A_{ij} = 1$ if nodes i and j are connected); $+I$: every node also keeps its own message
- $\tilde{D}$: degree matrix of $A + I$ (number of neighbours on the diagonal) - the normalisation keeps nodes with many neighbours from exploding
- $W^{(l)}$: the **shared** weights - the same for every node (like a conv filter)
- read it as: $\hat{A}$ = "average over the neighbours", W = "small linear layer", ReLU = activation
- **GAT**: replace the fixed weights in $\hat{A}$ by learned attention weights α_{ij}
- **graph-level** task (e.g. "is the molecule toxic?"): mean or sum over all node vectors → MLP head
- in PyTorch: the library **PyTorch Geometric** (`torch_geometric.nn.GCNConv`)

Kipf & Welling, ICLR 2017 · Veličković et al., ICLR 2018 · Tang et al., ICLR 2022 (GNNs for EEG seizure detection)

New applications

Deep learning in science and medicine - today

Science

- **proteins**: AlphaFold - 200 million shapes
- **weather**: GraphCast, GenCast - 10-15 days, in minutes on one chip
- **materials**: GNoME (2023) - 2.2 million new crystal structures predicted
- **maths and code**: LLMs as assistants for proofs and programming

Medicine

- **FDA** (USA): more than **1 400** AI-enabled medical devices cleared (end of 2025) - about 3/4 in **radiology**
- **segmentation**: SAM (2023) - "segment anything"; MedSAM for medical images
- **foundation models**: pathology, X-ray, EEG (e.g. LaBraM) - pre-train on huge unlabelled data
- **wearables**: ECG on a smartwatch, seizure alarms

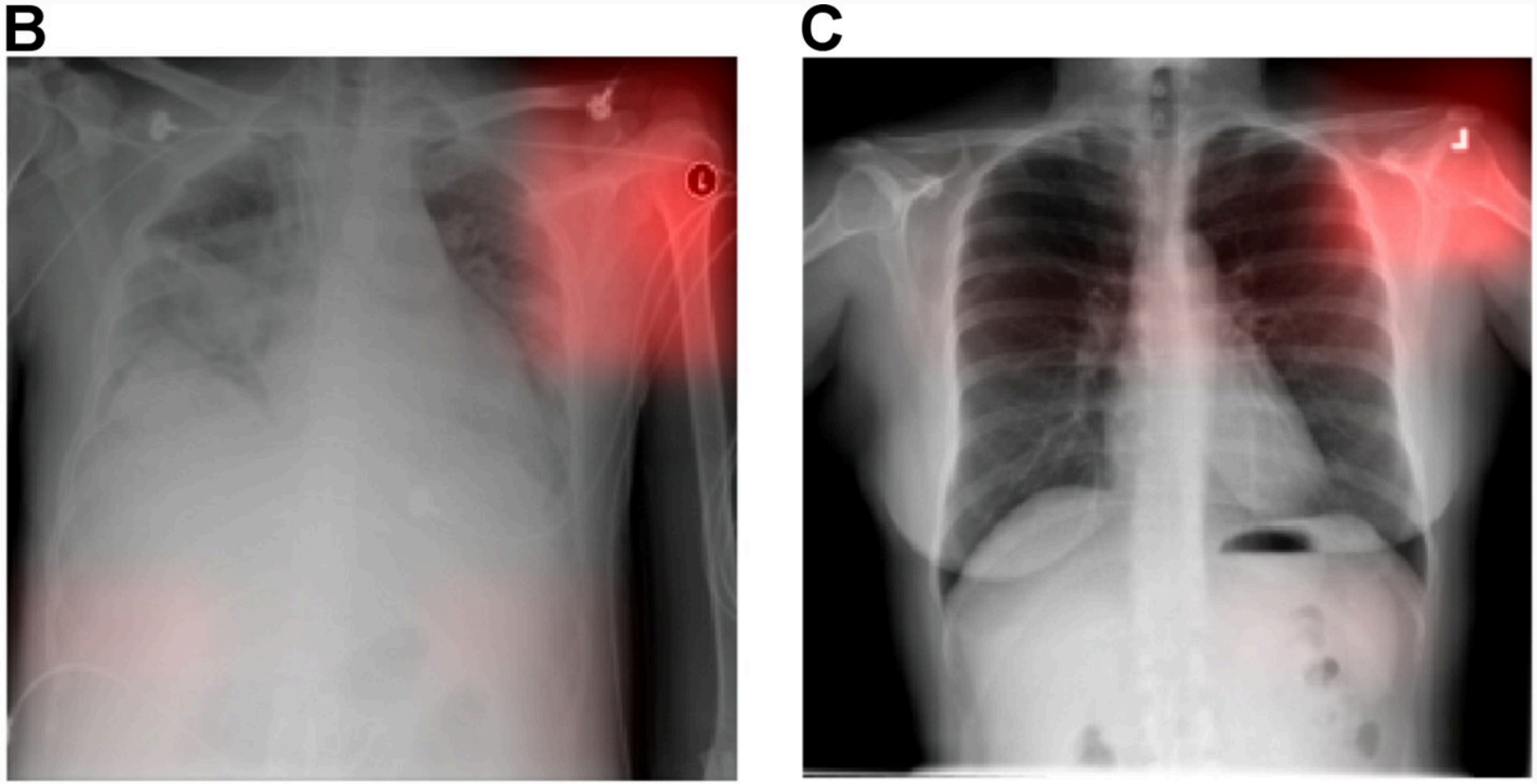
🌐 Try it (3 min)

aidemos.meta.com/segment-anything - click on any object in a photo: SAM draws its outline. No training for your object.

Lam et al., Science 2023 · Price et al., Nature 2025 (GenCast) · Merchant et al., Nature 2023 (GNoME) · FDA list of AI-enabled medical devices · Kirillov et al., ICCV 2023 · Ma et al., Nature Comm. 2024 (MedSAM) · Jiang et al., ICLR 2024 (LaBraM)

Responsible and explainable AI

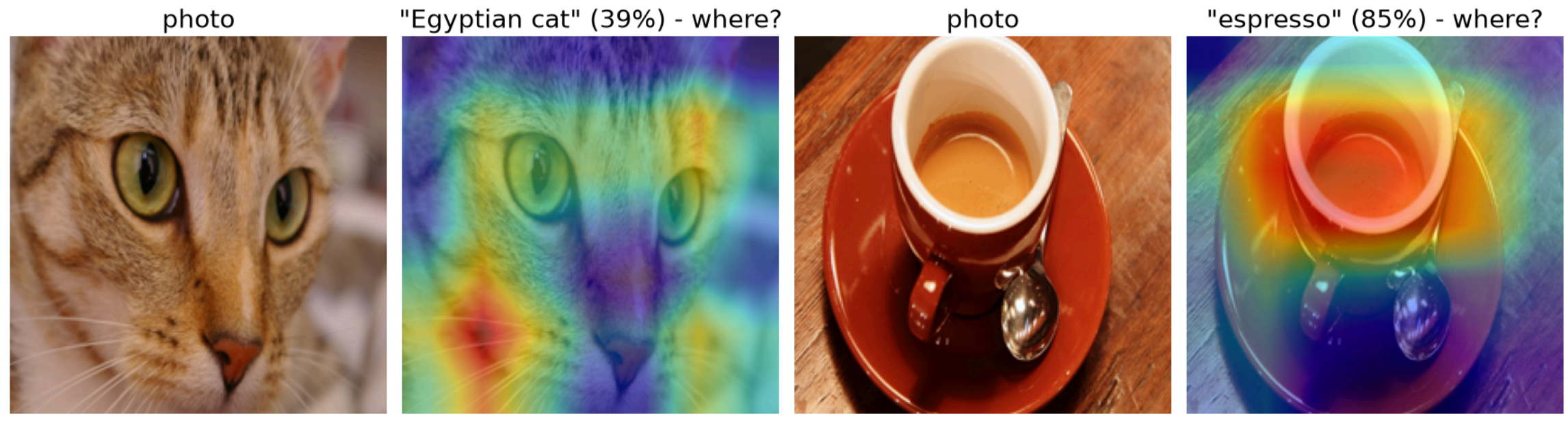
Where did the network look? A shortcut



A pneumonia CNN on chest X-rays: the red area is where it looked - the **metal marker of the hospital** (top right), not the lungs. Which hospital = how many sick patients.

Figure: Zech et al., "Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs", PLoS Medicine 2018, Fig 2 B+C (CC BY 4.0)

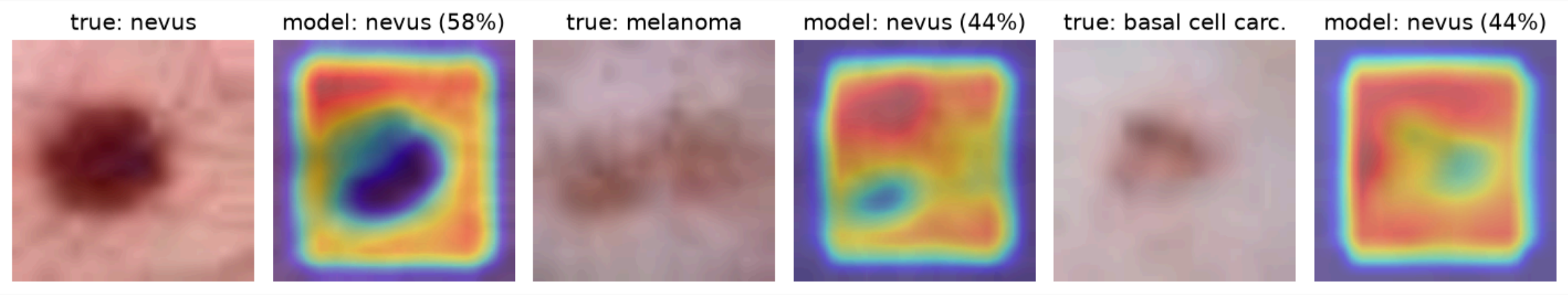
Explainable AI: Grad-CAM shows where a CNN looks



- **Grad-CAM**: how much does every region of the last feature maps push the score of the class? → a heatmap (red = important)
- a pretrained ResNet-18 (ImageNet): the cup and the coffee for "espresso", the face for "cat"
- a heatmap shows **where** - not **why**. It is a check, not a proof

Selvaraju et al., "Grad-CAM", ICCV 2017 · ResNet-18 from torchvision · photos: scikit-image sample data

Grad-CAM on our DermaMNIST baseline



- the baseline (two conv layers) says **"nevus"** for all three - the largest class (and for 96 % of all test pictures)
- and it looks at the **skin around** the lesion - not at the lesion
- that is why balanced accuracy, sensitivity per class - and a look at the heatmaps

🔑 Key takeaway

A good number + a heatmap on the right place = more trust. A good number alone is not enough.

Baseline model of the challenge · DermaMNIST test split (Yang et al. 2023, from HAM10000, Tschandl et al. 2018) · library for your own model: github.com/jacobgil/pytorch-grad-cam

The EU AI Act: rules by risk



forbidden: social scoring, manipulation

strict rules: **medical devices**, hiring, exams

must say “this is AI”: chatbots, deepfakes

no new rules: spam filter, games

- EU law, in force since **August 2024**, applies step by step
- since 2025: forbidden practices; rules for general-purpose models (LLMs)
- **AI in a medical device = high risk**: risk management, good data, documentation, **human oversight**, monitoring after release
- for medical devices from **August 2028** (postponed in 2026) - on top of the medical device law **MDR**

Regulation (EU) 2024/1689 (AI Act) · Regulation (EU) 2026/1744 (AI Omnibus) · Regulation (EU) 2017/745 (MDR) · general information, not legal advice

What stays

📉 The loop

loss, gradient, backprop, a step downhill - in every model since 1986

🧱 The building blocks

linear layer, convolution, memory, attention, shortcut, latent space - newer models **combine** them

🔍 Honest evaluation

train / validation / test, the right metric, external data - more important than the newest model

Your poster presentations 🎤

13:30-15:30

How it works

Every team

- poster full-screen on the big screen
- **15 minutes** - including your **live demo** (app)
- then **up to 5 minutes** of questions - everybody in the team must be able to answer
- the order is drawn at 13:30

Suggested story

1. problem + data (2 min)
2. baseline A → your improvements B (5 min)
3. results: **one** table, **one** figure (4 min)
4. live demo (2 min)
5. what did not work + one message (2 min)

Everyone else

- ask at least **one question** per team
- note one thing you liked and one idea for the team

Questions you might get

- How did you split the data? Was the test split used only once?
- Why balanced accuracy - and which class is the hardest?
- Is B really better than A - or is it within the noise?
- What did you try that did **not** work?
- Who did what in your team?

Leaderboard 🏆 and goodbye

15:30-16:00

The leaderboard: hidden test split

- every `model.pt` on the **official test split** - which no team has seen
- main metric: **balanced accuracy** (mean sensitivity of the 7 classes)
- also: accuracy, macro-F1, sensitivity per class
- **95 % interval** (bootstrap): if two intervals overlap a lot, the difference is probably **luck**

❓ Think about it

Team X has 0.71 [0.67, 0.75], team Y has 0.70 [0.66, 0.74]. Who is better?

The course on one slide

day	topic	the one idea
1	foundations, linear networks	a neuron = weighted sum; learn by gradient descent
2	MLP, backpropagation	layers + ReLU learn curves; backprop = chain rule backwards
3	training and evaluation	overfitting, regularisation, train / validation / test, the right metric
4	CNN	the same small filter everywhere; ResNet: the shortcut
5	RNN, LSTM, GRU	a memory along the time steps; gates against forgetting
6	Transformer, transfer learning	attention: every token looks at every token; pre-train, then fine-tune
7	autoencoder, VAE, GAN, diffusion	compress to a latent space; create new data
8	history, new building blocks, GNN	old ideas + data + GPUs; message passing on graphs

Hand-in by 15 November - on CampusWeb

- ☐ **link to your code** (Git repository) - with a README: how to run it
- ☐ **model** (`model.pt`)
- ☐ **report** (PDF, IEEE template, 2-4 pages)
- ☐ **poster** (PDF or PNG)
- ☐ **app** (link or code)

Last checks

- ☐ numbers in the report = numbers on the poster
- ☐ AI use statement + who did what
- ☐ limitations: data, test split, weak classes
- ☐ no data and no passwords in the repository

Your feedback

menti.com

the code is on the screen

Thank you! 🎓

Good luck with your hand-in - and with deep learning after this course.

Annika Stiehl · annika.stiehl@hs-ansbach.de · github.com/annikastiehl/srh-deep-learning